UNIVERSIDAD NACIONAL DE SAN AGUSTÍN DE AREQUIPA
FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS

LAB 4 · Framework Multiplataforma

MenúRadar

Prototipo móvil en Flutter para encontrar menús económicos y disponibles cerca del campus



Integrantes

Jorge Luis Escobedo Ocaña
Jose Alonzo Gordillo Mendoza

Docente

Norman Patrick Harvey Arce

Arequipa, Octubre de 2026

Índice de contenido

1. Descripción general del programa....4
1.1 Contexto y problema....4
1.2 Objetivo y alcance....4
1.3 Funcionalidades....5
1.4 Herramientas y versiones....5
2. Diseño de interfaces (funcionalidades y mockup)....6
2.1 Flujo general de la aplicación....7
2.2 Menús....8
2.3 Detalle del menú....9
2.4 Radar....10
2.5 Comunidad....11
2.6 Publicar menú....13
3. Descripción del proceso de desarrollo....14
3.1 Preparación del entorno....14
3.2 Estructura del proyecto....14
3.3 Cómo Flutter organiza el layout....15
3.4 Construcción de cada pantalla....15
3.5 Tipografía y tema....17
3.6 Ilustraciones con CustomPainter....18
3.7 Pruebas y control de versiones....19
4. Apreciación de la herramienta....20
4.1 Características a favor....20
4.2 Características en contra....20
Anexos: código de la aplicación....21
Referencias....32

Índice de imágenes

Figura 1. Árbol de problemas de MenúRadar (causas, problema central y efectos).....4
Figura 2. Mockup general: Menús, Detalle, Radar, Comunidad y Publicar menú.....6
Figura 3. Flujo general de la aplicación: navegación, acciones y estado compartido.....7
Figura 4. Pantalla Menús.....8
Figura 5. Menús al desplazar la lista.....8
Figura 6. Tarjeta de un menú agotado.....8
Figura 7. Detalle con la cabecera expandida.....9
Figura 8. Detalle con la cabecera colapsándose.....9
Figura 9. Radar sin selección.....10
Figura 10. Radar con un local seleccionado.....10
Figura 11. Comunidad con los avisos de ejemplo.....11
Figura 12. Comunidad tras publicar un aviso propio.....11
Figura 13. Hoja Nuevo aviso (primera vez, pide apodo).....12
Figura 14. Hoja Nuevo aviso con los datos completos.....12
Figura 15. Formulario al abrir.....13
Figura 16. Vista previa con nombre y precio.....13
Figura 17. Cocina «Otro» seleccionada.....13
Figura 18. Las cuatro ilustraciones: plato criollo, bowl vegetariano, bowl de chifa y cloche genérica.
.....18

Índice de tablas

Tabla 1. Funcionalidades del prototipo, la causa del problema que atacan y la pantalla que las contiene.....5
Tabla 2. Herramientas utilizadas y versiones comprobadas en el equipo de desarrollo.....5
Tabla 3. Opciones para identificar a quien publica un aviso en Comunidad.....11
Tabla 4. Organización de la carpeta `lib/` y responsabilidad de cada archivo.....14
Tabla 5. Widgets de organización y layout estudiados, su función y dónde se usaron.....15
Tabla 6. Tipografías del proyecto.....17
Tabla 7. Ilustración de cada cocina y las formas con que se dibuja.....18
Tabla 8. Verificaciones realizadas sobre el proyecto.....19

1. Descripción general del programa

1.1 Contexto y problema

MenúRadar es una aplicación móvil pensada para estudiantes universitarios. Su propósito es que puedan **comparar menús económicos y saber cuáles siguen disponibles** antes de salir del campus durante su receso. Parte de un trabajo previo del equipo desarrollado como aplicación web (PWA con React); en este laboratorio se rediseña la experiencia con **Flutter**, un framework multiplataforma, usando únicamente los componentes propios de la plataforma.

El diseño de las interfaces se deriva del siguiente árbol de problemas. La causa que más pesa es que la información de platos, precios y disponibilidad vive en pizarras físicas y cambia sin aviso, por lo que el estudiante solo se entera de que un plato se agotó cuando ya caminó hasta el local.

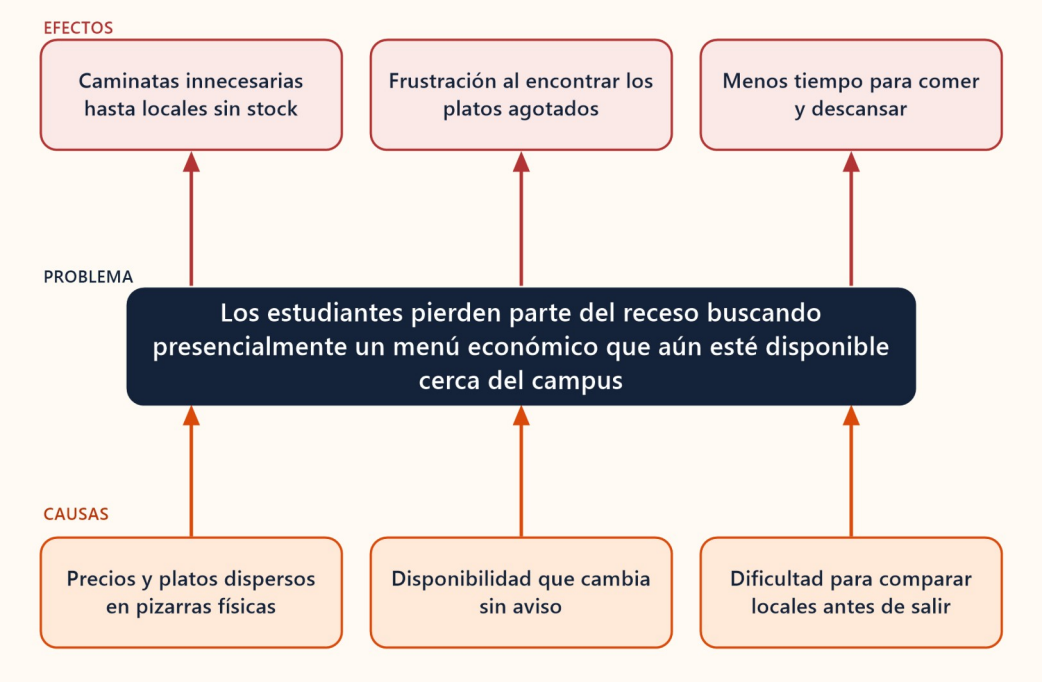


Figura 1. Árbol de problemas de MenúRadar (causas, problema central y efectos).

1.2 Objetivo y alcance

El objetivo es construir un prototipo funcional cuyas pantallas respondan a cada causa del árbol de problemas y que sirva para estudiar cómo Flutter organiza una interfaz y maneja el layout.

- **Incluido:** búsqueda y filtros de menús, detalle de cada menú, radar de locales, avisos de la comunidad, publicación de un menú y reporte de disponibilidad.
- **No incluido:** base de datos, cuentas de usuario, GPS, cámara ni sincronización entre dispositivos. Los locales y avisos iniciales son **datos de demostración** y los cambios viven en memoria mientras la app está abierta.
- **El radar es referencial:** representa los minutos a pie hasta cada local, no es un mapa geográfico.

1.3 Funcionalidades

Tabla 1. Funcionalidades del prototipo, la causa del problema que atacan y la pantalla que las contiene.

Funcionalidad	Causa que ataca	Pantalla
Buscar y filtrar menús por local, plato, precio o cocina	Información dispersa; difícil comparar	Menús
Ver precio, referencia, minutos a pie y platos de un menú	Información dispersa	Detalle
Ver y reportar si hay stock, quedan pocas raciones o se agotó	Disponibilidad sin aviso	Tarjeta, detalle y hoja inferior
Ver qué tan lejos está cada local en minutos a pie	Difícil comparar locales	Radar
Leer y publicar avisos de otros estudiantes	Disponibilidad sin aviso	Comunidad
Publicar un menú nuevo con vista previa	Información dispersa	Publicar menú

1.4 Herramientas y versiones

Tabla 2. Herramientas utilizadas y versiones comprobadas en el equipo de desarrollo.

Elemento	Detalle
Framework	Flutter 3.47.8 (canal stable)
Lenguaje	Dart 3.13.7
Diseño visual	Material 3 (`useMaterial3: true`) con tema propio
Paquetes externos en ejecución	Ninguno: solo el SDK de Flutter
Tipografías	Lilita One y Poppins, incluidas como recursos (licencia SIL OFL)
Dispositivo de prueba	Emulador Android «Pixel 9 Pro Dev» (Android 16, API 36)
Control de versiones	Git, con un commit por archivo modificado

2. Diseño de interfaces (funcionalidades y mockup)

Las cinco pantallas principales se muestran a continuación dentro de marcos de teléfono. Las capturas provienen del emulador Android, es decir, son la interfaz real construida con widgets y no maquetas dibujadas aparte.



Figura 2. Mockup general: Menús, Detalle, Radar, Comunidad y Publicar menú.

La navegación principal usa una barra inferior con tres destinos (**Menús**, **Radar** y **Comunidad**). Un botón flotante cambia según la pestaña: en Menús y Radar abre **Publicar menú**; en Comunidad abre **Nuevo aviso**. Desde una tarjeta o un pin se llega al **Detalle**.

2.1 Flujo general de la aplicación

El diagrama resume cómo se recorre la aplicación. La **barra de navegación** ofrece tres destinos y cada uno lleva a una acción propia: desde **Menús** se abre el detalle de una tarjeta; desde **Radar**, la vista previa de un local y desde ella el detalle; desde **Comunidad**, la hoja para publicar un aviso. El botón flotante **Publicar menú** abre el formulario y solo aparece en Menús y Radar.

Todas las acciones que cambian datos (avisar disponibilidad, publicar un menú o publicar un aviso) escriben en un único **estado compartido**, `MenuStore`, que es un `ChangeNotifier`. Las tres pestañas lo escuchan, así que un cambio hecho en una pantalla se ve de inmediato en las otras: la tarjeta cambia de estado, el pin del radar cambia de color y el aviso aparece primero en Comunidad. Esto es lo que hace coherente la experiencia sin necesidad de un servidor.

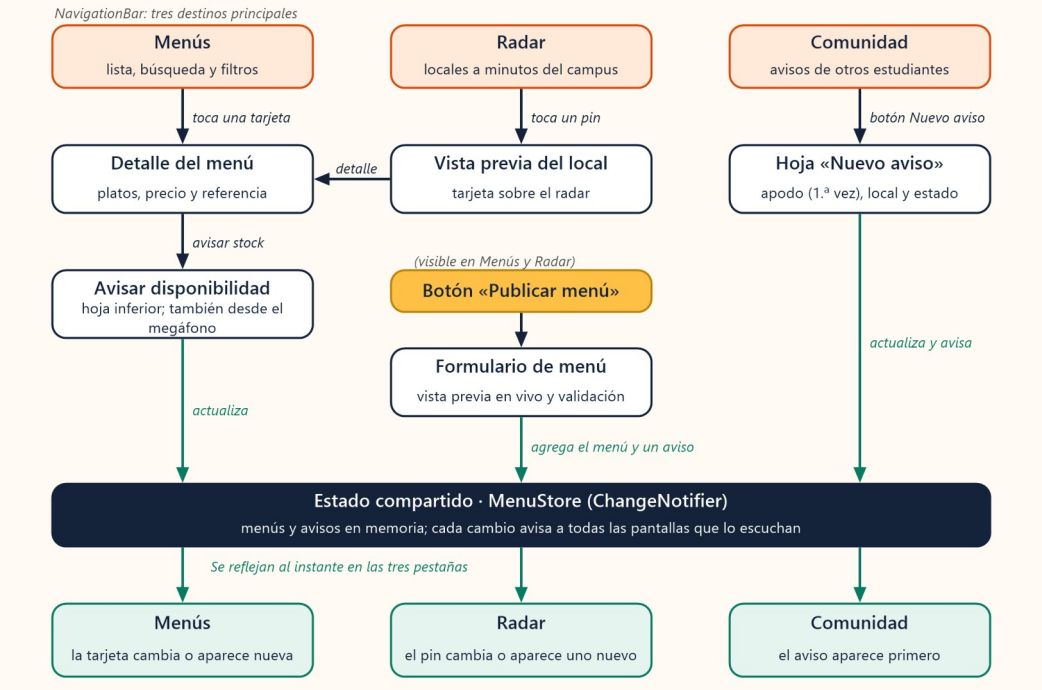


Figura 3. Flujo general de la aplicación: navegación, acciones y estado compartido.

Estos son los recorridos principales, en orden de uso:

- **Decidir dónde almorzar:** abrir Menús, filtrar o buscar y tocar una tarjeta para ver el detalle.
- **Evitar un viaje inútil:** tocar el megáfono de una tarjeta, o el botón del detalle, y elegir el estado que se observó.
- **Explorar por distancia:** abrir Radar, tocar un pin y entrar al detalle desde su vista previa.
- **Aportar información:** publicar un menú nuevo desde el botón flotante, o publicar un aviso desde Comunidad.

2.2 Menús

Es la pantalla de entrada. Responde a la pregunta «¿dónde almuerzo hoy?» con el mínimo de texto: cada tarjeta tiene una **ilustración del plato típico de la cocina**, el precio en grande y el estado de disponibilidad sobre la imagen. Debajo se muestran el nombre, tres datos con ícono (minutos a pie, referencia y cocina) y los platos de fondo como chips del color de la cocina.

- **Indicador de menús libres:** cápsula azul con un punto verde que late como el eco de un radar y el número en la tipografía de titulares.
- **Búsqueda y filtros:** un campo de texto y chips con ícono para filtrar por precio o por cocina vegetariana.
- **Menú agotado:** la imagen se oscurece y aparece un sello «AGOTADO» inclinado, de modo que el estado se reconoce sin leer.



Figura 4. Pantalla Menús.



Figura 5. Menús al desplazar la lista.



Figura 6. Tarjeta de un menú agotado.

2.3 Detalle del menú

Al tocar una tarjeta, la ilustración «viaja» hasta la cabecera del detalle con una animación compartida. La cabecera se **colapsa al desplazar**, dejando solo el nombre del local. El contenido se organiza en tres bloques: nombre con precio y estado, tres fichas de datos clave y una tarjeta con la entrada, el plato de fondo y la bebida. Un botón fijo en la parte inferior permite avisar la disponibilidad.



Figura 7. Detalle con la cabecera expandida.



Figura 8. Detalle con la cabecera colapsándose.

2.4 Radar

El radar convierte la lista de locales en una imagen que se entiende de un vistazo. En el centro está el campus; los anillos marcan **2, 4 y 6 minutos a pie** y cada local se ubica a la distancia que le corresponde. Un barrido animado recorre el círculo y el punto central emite ondas, lo que le da el carácter de radar. Cada pin tiene el color del estado y el ícono de la cocina, y su precio debajo. Al tocarlo crece, brilla y muestra una tarjeta con la imagen del plato que lleva al detalle.



Figura 9. Radar sin selección.



Figura 10. Radar con un local seleccionado.

2.5 Comunidad

Comunidad es la pantalla con el diseño más libre. La cabecera tiene un borde ondulado, círculos translúcidos, una ilustración girada y un sticker amarillo inclinado con el número de avisos. Cada aviso es una **burbuja de conversación** que alterna de lado y cuyo color depende del estado. Lleva una miniatura circular del plato, un avatar con la inicial de quien publica y un sello de estado ligeramente girado.

¿Cómo se publica un aviso? Se evaluaron tres formas de identificar a quien publica:

Tabla 3. Opciones para identificar a quien publica un aviso en Comunidad.

Opción	Ventaja	Inconveniente	Decisión
Cuenta con correo institucional	Autores verificables	Exige servidor y registro; frena el primer aviso	Para una versión real
Apodo local, sin cuenta	Se publica en segundos y no requiere servidor	No hay verificación	Usada en el prototipo
Anónimo total	Cero fricción	Los avisos no tienen quién los respalde	Descartada

La primera vez, la hoja **Nuevo aviso** pide un apodo y lo recuerda mientras la app está abierta. Después solo pide tres datos: **dónde**, **cómo está** y un comentario opcional. Publicar un aviso también actualiza el estado del menú elegido, así que la lista y el radar reflejan lo que reportó la comunidad.



Figura 11. Comunidad con los avisos de ejemplo.



Figura 12. Comunidad tras publicar un aviso propio.



Figura 13. Hoja Nuevo aviso (primera vez, pide apodo).



Figura 14. Hoja Nuevo aviso con los datos completos.

2.6 Publicar menú

El formulario se diseñó para que no parezca largo ni complejo. Arriba hay una **vista previa en vivo** del banner: cambia al elegir la cocina y al escribir el nombre y el precio. Los campos se agrupan en tres secciones numeradas (*El local*, *El menú de hoy* y *¿Cuánto queda?*), cada campo tiene un ícono y un ejemplo, y los errores aparecen solo en el campo que se está llenando.



Figura 15. Formulario al abrir.



Figura 16. Vista previa con nombre y precio.



Figura 17. Cocina «Otro» seleccionada.

3. Descripción del proceso de desarrollo

3.1 Preparación del entorno

Se instaló el SDK de Flutter 3.47.8 y se agregó su carpeta `bin` al Path de usuario de Windows. `flutter doctor` confirmó el soporte para Chrome y Android. Con `flutter create --project-name menu_radar --platforms=android,web .` se generaron las carpetas de plataforma sobre el código ya escrito.

Durante la preparación aparecieron dos obstáculos que conviene dejar documentados. Primero, el emulador existente tenía su partición de datos de 6 GB casi llena (94 %), por lo que el APK de depuración de 143 MB **no se pudo instalar** («not enough space»). Se resolvió creando un emulador nuevo con 16 GB de almacenamiento. Segundo, el SDK de Android no tenía las `cmdline-tools`; esto no impide compilar, porque Gradle descargó y aceptó las licencias del NDK, Build-Tools y CMake durante la primera compilación.

3.2 Estructura del proyecto

Tabla 4. Organización de la carpeta lib/ y responsabilidad de cada archivo.

Archivo	Responsabilidad
`main.dart`	Arranque de la app y `MaterialApp` con el tema.
`theme/app_theme.dart`	Colores, tipografías y `ThemeData` (campos, botones, tarjetas, barra de navegación).
`models/menu.dart`	Clases `CampusMenu`, `CommunityNotice` y el estado de disponibilidad.
`data/menu_store.dart`	Estado en memoria (`ChangeNotifier`): menús, avisos y apodo de sesión.
`screens/home_shell.dart`	`Scaffold`, `NavigationBar`, `IndexedStack` y botón flotante.
`screens/explore_tab.dart`	Pestaña Menús: encabezado, búsqueda, filtros y lista.
`screens/radar_tab.dart`	Pestaña Radar: lienzo animado, pines y vista previa.
`screens/community_tab.dart`	Pestaña Comunidad: cabecera ondulada y burbujas.
`screens/menu_detail_page.dart`	Detalle con `SliverAppBar` colapsable.
`screens/publish_menu_page.dart`	Formulario de publicación con vista previa.
`widgets/food_art.dart`	Ilustraciones de platos (`CustomPainter`) y banner por cocina.
`widgets/menu_card.dart`	Tarjeta de menú de la lista.
`widgets/status_badge.dart`	Indicador de disponibilidad.
`widgets/availability_sheet.dart`	Hoja inferior para reportar stock.
`widgets/notice_composer.dart`	Hoja inferior para publicar un aviso.

3.3 Cómo Flutter organiza el layout

Todo en Flutter es un widget, y el layout se resuelve con una regla simple: **las restricciones bajan, los tamaños suben y el padre decide la posición**. Cada widget recibe del padre un rango de tamaños permitido (sus restricciones), elige un tamaño dentro de ese rango y se lo comunica al padre, que lo ubica. Entender esto explica la mayoría de los errores de desborde. Por ejemplo, una `ListView` dentro de una `Column` falla si no se le acota la altura, y por eso en el proyecto siempre va dentro de un `Expanded`.

Tabla 5. Widgets de organización y layout estudiados, su función y dónde se usaron.

Widget	Función	Uso en MenúRadar
`Scaffold`	Estructura base: cuerpo, barra de navegación y botón flotante.	Todas las pantallas.
`SafeArea`	Evita invadir barras del sistema y recortes.	Shell y detalle.
`Column` / `Row`	Ordenan hijos en vertical u horizontal.	Encabezados, tarjetas, fichas de datos.
`Expanded`	Reparte el espacio libre del eje principal.	Lista de menús, panel del radar, fichas.
`Padding` / `SizedBox`	Márgenes y separaciones fijas.	Toda la app.
`Wrap`	Pasa a otra línea cuando falta ancho.	Datos y chips de la tarjeta; leyenda.
`ListView`	Contenido desplazable.	Menús, Comunidad, formulario, filtros.
`CustomScrollView` / `SliverAppBar`	Cabecera que se colapsa con el scroll.	Detalle del menú.
`Stack` / `Positioned`	Superponen y ubican elementos.	Banner de tarjeta, radar, cabecera.
`LayoutBuilder`	Entrega las restricciones recibidas para calcular medidas.	Posición de los pines del radar.
`IndexedStack`	Muestra un hijo y conserva el estado de los demás.	Pestañas del shell.
`NavigationBar`	Navegación principal de Material 3.	Menús, Radar, Comunidad.
`Hero`	Transición compartida entre pantallas.	Banner de la tarjeta al detalle.
`ClipRRect` / `ClipOval` / `ClipPath`	Recortan en formas.	Banner, miniaturas, onda de Comunidad.
`Transform.rotate`	Gira un widget.	Sellos y sticker.
`CustomPaint`	Dibujo propio sobre un lienzo.	Ilustraciones y radar.
`AnimatedSwitcher` / `AnimatedScale`	Animaciones implícitas.	Vista previa y pines del radar.

3.4 Construcción de cada pantalla

Menús

La pestaña es una `Column`: el encabezado y el buscador ocupan su alto natural, la fila de filtros es una `ListView` horizontal de altura fija, y la lista de tarjetas está dentro de un `Expanded` para recibir el espacio restante. Cada tarjeta es un `Card` con `InkWell`, de modo que toda la superficie es táctil.

El banner usa un `Stack`: la ilustración va al fondo y encima, con `Positioned`, el estado arriba a la izquierda y el precio abajo.

```
Scaffold
└─ SafeArea
   └─ IndexedStack
      └─ ExploreTab · Column
         ├─ Row (logo, título, indicador de libres)
         ├─ TextField
         ├─ ListView horizontal · FilterChip
         └─ Expanded
            └─ ListView.separated
               └─ MenuCard · Card
                  └─ InkWell · Column
                     ├─ Stack (Hero > FoodBanner, StatusBadge, precio)
                     └─ Padding (nombre, Wrap de datos, Wrap de chips)
```

Detalle

El detalle usa `CustomScrollView` con dos slivers. El `SliverAppBar` es `pinned` y tiene `expandedHeight`; dentro de su `flexibleSpace`, un `LayoutBuilder` calcula cuánto se ha colapsado para cambiar la opacidad del título. El contenido va en un `SliverToBoxAdapter`. Las tres fichas de datos son un `Row` con tres `Expanded`, por lo que reparten el ancho en partes iguales.

```
Scaffold
├─ body: CustomScrollView
│  ├─ SliverAppBar (pinned, expandedHeight 280)
│  │  └─ flexibleSpace: LayoutBuilder → Stack
│  │     ├─ Hero → FoodBanner (CustomPaint)
│  │     ├─ Opacity → Text (título al colapsar)
│  │     └─ Container (borde superior redondeado)
│  └─ SliverToBoxAdapter → Padding → Column
│     ├─ Text (nombre), Row (precio, StatusBadge)
│     ├─ Row → 3 × Expanded → _StatTile
│     ├─ Card → Column → 3 × _CourseRow
│     └─ Row (ícono + aviso)
└─ bottomNavigationBar: SafeArea → FilledButton.icon
```

Radar

El radar es un `StatefulWidget` con un `AnimationController` que dura cinco segundos y se repite. Un `LayoutBuilder` entrega el tamaño disponible; sobre él, un `Stack` superpone el lienzo (`CustomPaint`), el marcador del campus y un `Positioned` por cada local. Como el painter recibe la animación como `repaint`, redibuja el barrido en cada cuadro sin reconstruir los demás widgets.

```
RadarTab (StatefulWidget + SingleTickerProviderStateMixin)
└─ Padding → Column
   ├─ Row (títulos, cápsula con el número de locales)
   └─ Expanded → ClipRRect → LayoutBuilder → GestureDetector → Stack
      ├─ Positioned.fill → CustomPaint(_RadarPainter)
      ├─ Positioned → _CampusMarker
      ├─ Positioned × n → _RadarPin
      ├─ Positioned → _Legend (Wrap)
      └─ Positioned → AnimatedSwitcher → _Hint | _PreviewCard
```

La posición de cada pin se calcula con una fórmula propia: el **ángulo** sale de la posición ilustrativa del menú y la **distancia al centro** es proporcional a los minutos a pie. Se deja un radio mínimo para que ningún pin tape el marcador del campus. Los anillos del lienzo usan la misma función, de modo que pines y anillos siempre coinciden.

```
/// Fracción del radio del radar donde cae un local según sus minutos a pie.
/// Deja un espacio libre alrededor del marcador del campus.
double radarFraction(num minutes) =>
    0.22 + 0.78 * minutes.clamp(1, radarMaxMinutes) / radarMaxMinutes;
```

```
/// La distancia al centro equivale a los minutos a pie; el ángulo sale de la
/// posición ilustrativa del menú.
Offset _pinCenter(CampusMenu menu, Size size) {
  final angle = math.atan2(menu.positionY - 0.5, menu.positionX - 0.5);
```

```
  return size.center(Offset.zero) +
      Offset(math.cos(angle), math.sin(angle)) *
          (radarFraction(menu.walkMinutes) * _radarRadius(size));
}
```

Comunidad

La pantalla es un `ListView`. Su cabecera combina un `ClipPath` con un `CustomClipper` que recorta la onda inferior, un degradado, círculos dibujados con `CustomPaint`, una ilustración girada con `Transform.rotate` y un sticker. Las burbujas alternan el orden de sus hijos en un `Row` y la esquina redondeada cambia de lado según la posición.

```
class _WaveClipper extends CustomClipper<Path> {
  @override
  Path getClip(Size size) => Path()
    ..lineTo(0, size.height - 36)
    ..quadraticBezierTo(size.width * 0.25, size.height + 8, size.width * 0.52, size.height - 24)
    ..quadraticBezierTo(size.width * 0.80, size.height - 54, size.width, size.height - 14)
    ..lineTo(size.width, 0)
    ..close();

  @override
  bool shouldReclip(covariant CustomClipper<Path> oldClipper) => false;
}
```

Publicar menú

El formulario es un `Form` con un `ListView`. Cada campo es un `TextFormField` con `prefixIcon`, y un método auxiliar `_field` evita repetir la configuración. La vista previa escucha los controladores de nombre y precio con `ListenableBuilder`, así que se actualiza mientras se escribe. El selector de cocina son `ChoiceChip` dentro de una lista horizontal, y la disponibilidad un `SegmentedButton` dentro de un `Expanded` para que ocupe todo el ancho.

```
Scaffold → SafeArea → Form → ListView
├─ _Preview: SizedBox → FoodBanner → ListenableBuilder → Column
├─ SizedBox → ListView horizontal → ChoiceChip
├─ _Section(1) → TextFormField ×3 + Row[Expanded TextFormField ×2]
├─ _Section(2) → TextFormField ×3
├─ _Section(3) → Row → Expanded → SegmentedButton
└─ FilledButton.icon
```

La validación es por campo: cada `TextFormField` usa `AutovalidateMode.onUserInteraction`, de modo que un campo solo se marca en rojo cuando el usuario interactúa con él. Al pulsar *Publicar menú* se valida el formulario completo. Para que un menú nuevo no se encime con otro en el radar, `_freeAngle` prueba 24 ángulos y elige el más alejado de los pines que ya existen.

3.5 Tipografía y tema

Se incluyeron dos familias libres como recursos del proyecto, para que la app no dependa de internet. `ThemeData(fontFamily: ...)` fija Poppins como base y los estilos de títulos del tema usan Lilita One. Como Lilita One tiene un solo peso, nunca se le aplica negrita (la sintética se ve deformada); un método auxiliar `AppText.display()` entrega el estilo de titular.

Tabla 6. Tipografías del proyecto.

Familia	Uso	Motivo
Lilita One	Títulos, precios, nombres y sellos	Trazo grueso y redondeado, afín a un letrero de comida; da identidad con pocas palabras.
Poppins (Regular, Medium, SemiBold, Bold)	Lectura, etiquetas y botones	Geométrica y muy legible en pantallas pequeñas.

```
  /// Estilo de titular. Lilita One solo tiene un peso, por eso no se
  /// le asigna negrita.
  static TextStyle display(
    double size, {
    Color color = AppColors.navy,
    double height = 1.1,
    double letterSpacing = 0,
    List<Shadow>? shadows,
  }) => TextStyle(
    fontFamily: displayFamily,
    fontSize: size,
    color: color,
    height: height,
    letterSpacing: letterSpacing,
    fontWeight: FontWeight.w400,
    shadows: shadows,
  );
```

3.6 Ilustraciones con CustomPainter

Para que las tarjetas fueran visuales sin depender de fotos con licencia, los platos se dibujan con `CustomPainter` en un lienzo normalizado de 0 a 1 (`lib/widgets/food_art.dart`). Al escalar el canvas por el tamaño del widget, la misma ilustración sirve para el banner de una tarjeta, la cabecera del detalle o una miniatura de 56 píxeles. Cada cocina tiene su paleta de degradado y su plato.



Figura 18. Las cuatro ilustraciones: plato criollo, bowl vegetariano, bowl de chifa y cloche genérica.

Tabla 7. Ilustración de cada cocina y las formas con que se dibuja.

Cocina	Elementos dibujados
Criollo	Plato circular con arroz (óvalo con granos), papas fritas y tiras de carne (rectángulos redondeados girados), cebolla (arcos) y tomate.
Vegetariano	Bowl formado por una elipse y un arco, hojas (elipses giradas), tomates y zanahoria (círculos).
Chifa	Bowl rojo, fideos (curvas de Bézier), cebollín (círculos), palillos (líneas) y vapor.
Otro	Cloche: semicírculo con brillo, base, perilla y destellos en cruz.

```
void _bar(
  Canvas canvas,
  Offset center,
  double width,
  double height,
```

```
  double angle,
  Color color,
) {
  canvas
    ..save()
    ..translate(center.dx, center.dy)
    ..rotate(angle)
    ..drawRRect(
      RRect.fromRectAndRadius(
        Rect.fromCenter(center: Offset.zero, width: width, height: height),
        Radius.circular(math.min(width, height) * 0.45),
      ),
      _fill(color),
    )
    ..restore();
}
```

3.7 Pruebas y control de versiones

Tabla 8. Verificaciones realizadas sobre el proyecto.

Verificación	Resultado
`flutter analyze`	Sin avisos ni errores.
`flutter test`	Prueba de widget superada: la app arranca y muestra la navegación.
`flutter build apk --debug`	Compila (APK de 143 MB).
`flutter build apk --release --target-platform android-x64`	Compila (APK de 17,8 MB).
Ejecución en el emulador	Se recorrieron las cinco pantallas, el reporte de disponibilidad, la publicación de un aviso y la validación del formulario.

El código se versionó con Git haciendo **un commit por archivo modificado**, con mensajes que describen el cambio de ese archivo. Esto deja un historial de más de cien commits que permite seguir cómo evolucionó cada pantalla.

4. Apreciación de la herramienta

Tras construir el prototipo, estas son dos características de Flutter a favor y dos en contra, basadas en lo que ocurrió durante el desarrollo.

4.1 Características a favor

1. Ciclo de desarrollo rápido gracias a Hot Reload. Se pudieron probar cambios de layout, colores y tipografía en el emulador casi al instante, sin reiniciar la app. Esto fue decisivo en el rediseño, donde se ajustaron márgenes, tamaños de letra y posiciones de pines varias veces hasta que la pantalla se vio como se esperaba.

2. Control total sobre la interfaz. Flutter dibuja sus propios widgets en lugar de apoyarse en los controles nativos, y ofrece `CustomPainter` para dibujar cualquier forma. Con eso se crearon las ilustraciones de los platos, el radar con su barrido animado y la onda de Comunidad sin imágenes externas ni librerías, y el resultado se ve igual en cualquier dispositivo.

4.2 Características en contra

1. Código muy anidado y extenso. Una interfaz se describe como un árbol de widgets, y una pantalla sencilla puede superar los diez niveles de sangría. Un archivo como `radar_tab.dart` llega a más de 450 líneas, y para mantenerlo legible hubo que dividir las pantallas en widgets privados. Esto exige disciplina y cierta práctica para no perderse en el árbol.

2. Entorno de desarrollo pesado. Preparar Flutter para Android exige el SDK, el emulador, las licencias y la configuración de Gradle, y consume mucho espacio y tiempo. En el proyecto, un emulador sin espacio impidió instalar el APK de depuración de 143 MB, y la primera compilación tardó varios minutos. El tamaño de la app de depuración es también mucho mayor que el de la versión final (17,8 MB).

Anexos: código de la aplicación

Se incluye únicamente el código más representativo: el arranque, el modelo de datos, la navegación principal y los widgets que muestran cómo se organiza el layout. En los archivos largos se transcribe solo la clase o el método relevante. El código completo está en el repositorio del proyecto (carpeta `lib/`).

Anexo A. Arranque de la aplicación y tema (lib/main.dart)

```
import 'package:flutter/material.dart';

import 'data/menu_store.dart';
import 'screens/home_shell.dart';
import 'theme/app_theme.dart';

void main() => runApp(const MenuRadarApp());

class MenuRadarApp extends StatefulWidget {
  const MenuRadarApp({super.key});

  @override
  State<MenuRadarApp> createState() => _MenuRadarAppState();
}

class _MenuRadarAppState extends State<MenuRadarApp> {
  final MenuStore _store = MenuStore();

  @override
  void dispose() {
    _store.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) => MaterialApp(
    title: 'MenúRadar',
    debugShowCheckedModeBanner: false,
    theme: buildAppTheme(),
    home: HomeShell(store: _store),
  );
}
```

Anexo B. Modelo de datos y fórmula del radar (lib/models/menu.dart)

```
enum MenuAvailability { available, limited, soldOut }

extension MenuAvailabilityLabel on MenuAvailability {
  String get label => switch (this) {
    MenuAvailability.available => 'Disponible',
    MenuAvailability.limited => 'Últimas raciones',
    MenuAvailability.soldOut => 'Agotado',
  };

  String get noticeText => switch (this) {
    MenuAvailability.available => 'Hay raciones disponibles.',
    MenuAvailability.limited => 'Quedan pocas raciones.',
    MenuAvailability.soldOut => 'Se agotó por ahora.',
  };
}

/// Minutos a pie que representa el anillo exterior del radar.
const int radarMaxMinutes = 6;

/// Fracción del radio del radar donde cae un local según sus minutos a pie.
/// Deja un espacio libre alrededor del marcador del campus.
double radarFraction(num minutes) =>
    0.22 + 0.78 * minutes.clamp(1, radarMaxMinutes) / radarMaxMinutes;

class CampusMenu {
  const CampusMenu({
    required this.id,
```

```
    required this.restaurant,
    required this.price,
    required this.walkMinutes,
    required this.area,
    required this.category,
    required this.starters,
    required this.mainDishes,
    required this.drink,
    required this.availability,
    required this.positionX,
    required this.positionY,
  });

  final String id;
  final String restaurant;
  final double price;
  final int walkMinutes;
  final String area;
  final String category;
  final List<String> starters;
  final List<String> mainDishes;
  final String drink;
  final MenuAvailability availability;
  final double positionX;
  final double positionY;

  CampusMenu copyWith({MenuAvailability? availability}) => CampusMenu(
    id: id,
    restaurant: restaurant,
    price: price,
    walkMinutes: walkMinutes,
    area: area,
    category: category,
    starters: starters,
    mainDishes: mainDishes,
    drink: drink,
    availability: availability ?? this.availability,
    positionX: positionX,
    positionY: positionY,
  );
}

class CommunityNotice {
  const CommunityNotice({
    required this.author,
    required this.restaurant,
    required this.category,
    required this.message,
    required this.availability,
    required this.createdAt,
  });

  final String author;
  final String restaurant;
  final String category;
  final String message;
  final MenuAvailability availability;
  final DateTime createdAt;
}
```

Anexo C. Estado compartido: reporte de disponibilidad y publicación (lib/data/menu_store.dart, extracto)

```
  /// Actualiza el estado de un menú y publica un aviso. Si [comment] viene
  /// vacío se usa el texto estándar del estado.
  void updateAvailability(
    String id,
    MenuAvailability availability, {
    String? comment,
  }) {
    final index = _menus.indexWhere((menu) => menu.id == id);
    final text = comment?.trim() ?? '';
    if (index < 0) return;
    if (_menus[index].availability == availability && text.isEmpty) return;
    final current = _menus[index];
    _menus[index] = current.copyWith(availability: availability);
    _notices.insert(
      0,
      CommunityNotice(
        author: _author,
        restaurant: current.restaurant,
```

```
        category: current.category,
        message: text.isEmpty ? availability.noticeText : text,
        availability: availability,
        createdAt: DateTime.now(),
      ),
    );
    notifyListeners();
  }

  void addMenu(CampusMenu menu) {
    _menus.insert(0, menu);
    _notices.insert(
      0,
      CommunityNotice(
        author: _author,
        restaurant: menu.restaurant,
        category: menu.category,
        message: 'Publicó un menú por S/ ${menu.price.toStringAsFixed(2)}.',
        availability: menu.availability,
        createdAt: DateTime.now(),
      ),
    );
    notifyListeners();
  }
}
```

Anexo D. Navegación principal y botón flotante (lib/screens/home_shell.dart)

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';

import '../data/menu_store.dart';
import '../models/menu.dart';
import '../widgets/availability_sheet.dart';
import '../widgets/notice_composer.dart';
import 'community_tab.dart';
import 'explore_tab.dart';
import 'menu_detail_page.dart';
import 'publish_menu_page.dart';
import 'radar_tab.dart';

class HomeShell extends StatefulWidget {
  const HomeShell({super.key, required this.store});

  final MenuStore store;

  @override
  State<HomeShell> createState() => _HomeShellState();
}

class _HomeShellState extends State<HomeShell> {
  int _selectedIndex = 0;

  Future<void> _openMenu(CampusMenu menu) async {
    await Navigator.push<void>(
      context,
      MaterialPageRoute(
        builder: (_) => MenuDetailPage(store: widget.store, menuId: menu.id),
      ),
    );
  }

  Future<void> _publishMenu() async {
    final id = await Navigator.push<String>(
      context,
      MaterialPageRoute(builder: (_) => PublishMenuPage(store: widget.store)),
    );
    if (id == null || !mounted) return;
    setState(() => _selectedIndex = 0);
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Menú publicado en esta sesión.')),
    );
  }

  Future<void> _newNotice() async {
    final published = await showNoticeComposer(context, widget.store);
    if (!published || !mounted) return;
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Aviso publicado en esta sesión.')),
    );
  }
```

```
  @override
  Widget build(BuildContext context) => Scaffold(
    // Comunidad dibuja su cabecera hasta el borde superior de la pantalla,
    // por eso allí la barra de estado lleva íconos claros.
    body: AnnotatedRegion<SystemUiOverlayStyle>(
      value: (_selectedIndex == 2
              ? SystemUiOverlayStyle.light
              : SystemUiOverlayStyle.dark)
          .copyWith(statusBarColor: Colors.transparent),
      child: SafeArea(
        top: _selectedIndex != 2,
        child: IndexedStack(
          index: _selectedIndex,
          children: [
            ExploreTab(
              store: widget.store,
              onOpen: _openMenu,
              onReport: (menu) => reportAvailability(context, widget.store, menu),
            ),
            RadarTab(store: widget.store, onOpen: _openMenu),
            CommunityTab(store: widget.store),
          ],
        ),
      ),
    ),
    floatingActionButton: FloatingActionButton.extended(
      onPressed: _selectedIndex == 2 ? _newNotice : _publishMenu,
      icon: Icon(_selectedIndex == 2 ? Icons.edit_note_rounded : Icons.add_rounded),
      label: Text(_selectedIndex == 2 ? 'Nuevo aviso' : 'Publicar menú'),
    ),
    bottomNavigationBar: NavigationBar(
      selectedIndex: _selectedIndex,
      onDestinationSelected: (index) => setState(() => _selectedIndex = index),
      destinations: const [
        NavigationDestination(
          icon: Icon(Icons.restaurant_menu_outlined),
          selectedIcon: Icon(Icons.restaurant_menu),
          label: 'Menús',
        ),
        NavigationDestination(
          icon: Icon(Icons.radar_outlined),
          selectedIcon: Icon(Icons.radar),
          label: 'Radar',
        ),
        NavigationDestination(
          icon: Icon(Icons.forum_outlined),
          selectedIcon: Icon(Icons.forum),
          label: 'Comunidad',
        ),
      ],
    ),
  );
}
```

Anexo E. Tarjeta de menú (Card, Stack, Wrap) (lib/widgets/menu_card.dart, extracto)

```
class MenuCard extends StatelessWidget {
  const MenuCard({
    super.key,
    required this.menu,
    required this.onOpen,
    required this.onReport,
  });

  final CampusMenu menu;
  final VoidCallback onOpen;
  final VoidCallback onReport;

  @override
  Widget build(BuildContext context) {
    final style = categoryStyle(menu.category);
    final soldOut = menu.availability == MenuAvailability.soldOut;
    return Card(
      clipBehavior: Clip.antiAlias,
      margin: EdgeInsets.zero,
      child: InkWell(
        onTap: onOpen,
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
```

```
          children: [
            Stack(
              children: [
                Hero(
                  tag: 'banner-${menu.id}',
                  child: FoodBanner(category: menu.category, height: 156),
                ),
                if (soldOut) const Positioned.fill(child: _SoldOutVeil()),
                Positioned(
                  top: 12,
                  left: 12,
                  child: StatusBadge(status: menu.availability, solid: true),
                ),
                Positioned(
                  left: 16,
                  bottom: 12,
                  child: Text(
                    'S/ ${menu.price.toStringAsFixed(2)}',
                    style: AppText.display(
                      36,
                      color: Colors.white,
                      shadows: const [Shadow(color: Color(0x55000000), blurRadius: 8)],
                    ),
                  ),
                ),
              ],
            ),
            Padding(
              padding: const EdgeInsets.fromLTRB(16, 14, 8, 14),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Row(
                    children: [
                      Expanded(
                        child: Text(
                          menu.restaurant,
                          maxLines: 1,
                          overflow: TextOverflow.ellipsis,
                          style: Theme.of(context).textTheme.titleLarge?.copyWith(
                            color: AppColors.navy,
                            fontSize: 23,
                          ),
                        ),
                      ),
                      IconButton.filledTonal(
                        tooltip: 'Avisar stock',
                        onPressed: onReport,
                        icon: const Icon(Icons.campaign_rounded),
                      ),
                    ],
                  ),
                  Padding(
                    padding: const EdgeInsets.only(right: 8),
                    child: Wrap(
                      spacing: 14,
                      runSpacing: 6,
                      children: [
                        _Meta(icon: Icons.directions_walk_rounded, text: '${menu.walkMinutes} min'),
                        _Meta(icon: Icons.place_outlined, text: menu.area),
                        _Meta(icon: categoryIcon(menu.category), text: menu.category),
                      ],
                    ),
                  ),
                  const SizedBox(height: 12),
                  Padding(
                    padding: const EdgeInsets.only(right: 8),
                    child: Wrap(
                      spacing: 8,
                      runSpacing: 8,
                      children: [
                        for (final dish in menu.mainDishes)
                          Container(
                            padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
                            decoration: BoxDecoration(
                              color: style.top.withAlpha(30),
                              borderRadius: BorderRadius.circular(100),
                            ),
                            child: Text(
                              dish,
                              style: TextStyle(
                                color: style.bottom,
                                fontWeight: FontWeight.w700,
                                fontSize: 13,
                              ),
                            ),
```

```
                        ),
                      ],
                    ),
                  ),
                ],
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

Anexo F. Dibujo del radar con CustomPainter (lib/screens/radar_tab.dart, extracto)

```
class _RadarPainter extends CustomPainter {
  _RadarPainter(this.animation) : super(repaint: animation);

  final Animation<double> animation;

  static const _glow = Color(0xFFFF8A3D);

  @override
  void paint(Canvas canvas, Size size) {
    final center = size.center(Offset.zero);
    final radius = _radarRadius(size);
    canvas.drawRect(
      Offset.zero & size,
      Paint()
        ..shader = RadialGradient(
          colors: const [Color(0xFF26507A), Color(0xFF0C192B)],
        ).createShader(Rect.fromCircle(center: center, radius: size.longestSide * 0.7)),
    );

    final line = Paint()
      ..color = Colors.white.withAlpha(46)
      ..style = PaintingStyle.stroke
      ..strokeWidth = 1;
    canvas
      ..drawLine(Offset(center.dx, center.dy - radius - 14), Offset(center.dx, center.dy + radius +
14), line)
      ..drawLine(Offset(center.dx - radius - 14, center.dy), Offset(center.dx + radius + 14,
center.dy), line);
    for (final minutes in const [2, 4, radarMaxMinutes]) {
      final r = radius * radarFraction(minutes);
      canvas.drawCircle(center, r, line);
      _label(canvas, '$minutes min', Offset(center.dx + 6, center.dy - r + 3));
    }

    final angle = animation.value * 2 * math.pi;
    canvas
      ..drawCircle(
        center,
        radius,
        Paint()
          ..shader = SweepGradient(
            colors: const [Color(0x00FF8A3D), Color(0x00FF8A3D), Color(0x70FF8A3D)],
            stops: const [0, 0.7, 1],
            transform: GradientRotation(angle),
          ).createShader(Rect.fromCircle(center: center, radius: radius)),
      )
      ..drawLine(
        center,
        center + Offset(math.cos(angle), math.sin(angle)) * radius,
        Paint()
          ..color = _glow.withAlpha(190)
          ..strokeWidth = 2
          ..strokeCap = StrokeCap.round,
      );

    final pulse = (animation.value * 3) % 1;
    canvas.drawCircle(
      center,
      22 + pulse * 46,
      Paint()
        ..color = _glow.withAlpha(((1 - pulse) * 150).round())
        ..style = PaintingStyle.stroke
        ..strokeWidth = 2,
    );
  }
```

```
  void _label(Canvas canvas, String text, Offset offset) {
    final painter = TextPainter(
      text: TextSpan(
        text: text,
        style: TextStyle(
          color: Colors.white.withAlpha(140),
          fontSize: 11,
          fontWeight: FontWeight.w700,
        ),
      ),
      textDirection: TextDirection.ltr,
    )..layout();
    painter.paint(canvas, offset);
  }

  @override
  bool shouldRepaint(covariant _RadarPainter oldDelegate) => false;
}
```

Anexo G. Detalle con SliverAppBar colapsable (lib/screens/menu_detail_page.dart, extracto)

```
class MenuDetailPage extends StatelessWidget {
  const MenuDetailPage({super.key, required this.store, required this.menuId});

  final MenuStore store;
  final String menuId;

  static const double _expandedHeight = 280;

  @override
  Widget build(BuildContext context) => AnimatedBuilder(
    animation: store,
    builder: (context, _) {
      final menu = store.findMenu(menuId);
      final style = categoryStyle(menu.category);
      final top = MediaQuery.paddingOf(context).top;
      return Scaffold(
        body: CustomScrollView(
          slivers: [
            SliverAppBar(
              pinned: true,
              expandedHeight: _expandedHeight,
              backgroundColor: style.bottom,
              foregroundColor: Colors.white,
              automaticallyImplyLeading: false,
              leading: Padding(
                padding: const EdgeInsets.all(6),
                child: IconButton(
                  style: IconButton.styleFrom(
                    backgroundColor: Colors.white.withAlpha(70),
                    foregroundColor: Colors.white,
                  ),
                  icon: const Icon(Icons.arrow_back_rounded),
                  onPressed: () => Navigator.pop(context),
                ),
              ),
              flexibleSpace: LayoutBuilder(
                builder: (context, constraints) {
                  final collapsed = kToolbarHeight + top;
                  final t = ((constraints.maxHeight - collapsed) /
                          (_expandedHeight + top - collapsed))
                      .clamp(0.0, 1.0);
                  return Stack(
                    fit: StackFit.expand,
                    children: [
                      Hero(
                        tag: 'banner-${menu.id}',
                        child: FoodBanner(category: menu.category),
                      ),
                      Positioned(
                        top: top + 15,
                        left: 64,
                        right: 16,
                        child: Opacity(
                          opacity: 1 - t,
                          child: Text(
                            menu.restaurant,
                            maxLines: 1,
                            overflow: TextOverflow.ellipsis,
                            style: AppText.display(22, color: Colors.white),
```

```
                    ),
                  ),
                ),
                Positioned(
                  left: 0,
                  right: 0,
                  bottom: -1,
                  child: Container(
                    height: 28,
                    decoration: const BoxDecoration(
                      color: AppColors.cream,
                      borderRadius: BorderRadius.vertical(top: Radius.circular(28)),
                    ),
                  ),
                ),
              ],
            );
          },
        ),
      ),
      SliverToBoxAdapter(
        child: Padding(
          padding: const EdgeInsets.fromLTRB(20, 4, 20, 24),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(
                menu.restaurant,
                style: Theme.of(context).textTheme.headlineMedium?.copyWith(
                  color: AppColors.navy,
                  fontSize: 34,
                ),
              ),
              const SizedBox(height: 6),
              Row(
                crossAxisAlignment: CrossAxisAlignment.center,
                children: [
                  Text(
                    'S/ ${menu.price.toStringAsFixed(2)}',
                    style: AppText.display(36, color: AppColors.orange),
                  ),
                  const SizedBox(width: 12),
                  StatusBadge(status: menu.availability),
                ],
              ),
              const SizedBox(height: 18),
              Row(
                children: [
                  Expanded(
                    child: _StatTile(
                      icon: Icons.directions_walk_rounded,
                      value: '${menu.walkMinutes} min',
                      label: 'a pie',
                      color: style.top,
                    ),
                  ),
                  const SizedBox(width: 10),
                  Expanded(
                    child: _StatTile(
                      icon: Icons.place_rounded,
                      value: menu.area,
                      label: 'referencia',
                      color: style.top,
                    ),
                  ),
                  const SizedBox(width: 10),
                  Expanded(
                    child: _StatTile(
                      icon: categoryIcon(menu.category),
                      value: menu.category,
                      label: 'cocina',
                      color: style.top,
                    ),
                  ),
                ],
              ),
              const SizedBox(height: 20),
              Card(
                margin: EdgeInsets.zero,
                child: Padding(
                  padding: const EdgeInsets.all(18),
                  child: Column(
                    children: [
                      _CourseRow(
                        icon: Icons.soup_kitchen_rounded,
                        title: 'Entrada',
```

```
                            items: menu.starters,
                            color: style.top,
                          ),
                          const _Divider(),
                          _CourseRow(
                            icon: Icons.restaurant_rounded,
                            title: 'Plato de fondo',
                            items: menu.mainDishes,
                            color: style.top,
                          ),
                          const _Divider(),
                          _CourseRow(
                            icon: Icons.local_drink_rounded,
                            title: 'Bebida',
                            items: [menu.drink],
                            color: style.top,
                          ),
                        ],
                      ),
                    ),
                  ),
                  const SizedBox(height: 14),
                  Row(
                    children: [
                      const Icon(Icons.info_outline_rounded, size: 16, color: Colors.black45),
                      const SizedBox(width: 6),
                      Expanded(
                        child: Text(
                          'Confirma precio y stock en el local.',
                          style: Theme.of(context).textTheme.bodySmall,
                        ),
                      ),
                    ],
                  ),
                ],
              ),
            ),
          ),
        ],
      ),
      bottomNavigationBar: SafeArea(
        minimum: const EdgeInsets.fromLTRB(20, 8, 20, 16),
        child: FilledButton.icon(
          onPressed: () => reportAvailability(context, store, menu),
          icon: const Icon(Icons.campaign_rounded),
          label: const Text('Avisar disponibilidad'),
        ),
      ),
    );
  },
);
}
```

Anexo H. Banner con degradado e ilustración (lib/widgets/food_art.dart, extracto)

```
class FoodBanner extends StatelessWidget {
  const FoodBanner({
    super.key,
    required this.category,
    this.height,
    this.radius = BorderRadius.zero,
    this.child,
  });

  final String category;
  final double? height;
  final BorderRadius radius;
  final Widget? child;

  @override
  Widget build(BuildContext context) {
    final style = categoryStyle(category);
    return ClipRRect(
      borderRadius: radius,
      child: SizedBox(
        height: height,
        width: double.infinity,
        child: Stack(
          fit: StackFit.expand,
          children: [
            DecoratedBox(
              decoration: BoxDecoration(
                gradient: LinearGradient(
                  begin: Alignment.topLeft,
```

```
                  end: Alignment.bottomRight,
                  colors: [style.top, style.bottom],
                ),
              ),
            ),
            CustomPaint(painter: _BackdropPainter(style.accent)),
            LayoutBuilder(
              builder: (context, constraints) {
                final side =
                    math.min(constraints.maxHeight, constraints.maxWidth) * 1.1;
                return Stack(
                  children: [
                    Positioned(
                      right: -side * 0.06,
                      bottom: -side * 0.14,
                      width: side,
                      height: side,
                      child: FoodArt(category: category),
                    ),
                  ],
                );
              },
            ),
            if (child != null) child!,
          ],
        ),
      ),
    );
  }
}
```

Anexo I. Vista previa en vivo del formulario (lib/screens/publish_menu_page.dart, extracto)

```
/// Vista previa en vivo del banner que verán los demás estudiantes.
class _Preview extends StatelessWidget {
  const _Preview({
    required this.category,
    required this.restaurant,
    required this.price,
  });

  final String category;
  final TextEditingController restaurant;
  final TextEditingController price;

  @override
  Widget build(BuildContext context) => SizedBox(
    height: 150,
    child: FoodBanner(
      category: category,
      radius: BorderRadius.circular(28),
      child: ListenableBuilder(
        listenable: Listenable.merge([restaurant, price]),
        builder: (context, _) {
          final name = restaurant.text.trim();
          final value = price.text.trim();
          return Padding(
            padding: const EdgeInsets.fromLTRB(18, 14, 18, 14),
            child: Column(
              mainAxisAlignment: MainAxisAlignment.end,
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  'Así se verá',
                  style: TextStyle(
                    color: Colors.white.withAlpha(200),
                    fontSize: 12,
                    fontWeight: FontWeight.w600,
                  ),
                ),
                Text(
                  value.isEmpty ? 'S/ --' : 'S/ $value',
                  style: AppText.display(34, color: Colors.white),
                ),
                SizedBox(
                  width: 190,
                  child: Text(
                    name.isEmpty ? 'Tu local' : name,
                    maxLines: 1,
                    overflow: TextOverflow.ellipsis,
                    style: AppText.display(18, color: Colors.white),
                  ),
```

```
                ),
              ],
            ),
          );
        },
      ),
    ),
  );
}
```

Anexo J. Apertura de la hoja Nuevo aviso (lib/widgets/notice_composer.dart, extracto)

```
/// Abre la hoja para publicar un aviso en Comunidad. Devuelve `true` si se
/// publicó.
Future<bool> showNoticeComposer(BuildContext context, MenuStore store) async {
  final published = await showModalBottomSheet<bool>(
    context: context,
    isScrollControlled: true,
    showDragHandle: true,
    builder: (_) => _NoticeComposer(store: store),
  );
  return published ?? false;
}
```

Anexo K. Prueba de widget (test/widget_test.dart)

```
import 'package:flutter_test/flutter_test.dart';

import 'package:menu_radar/main.dart';

void main() {
  testWidgets('La app arranca y muestra la barra de navegación', (tester) async {
    await tester.pumpWidget(const MenuRadarApp());

    expect(find.text('Radar'), findsWidgets);
    expect(find.text('Comunidad'), findsWidgets);
  });
}
```

Referencias

Flutter. (s. f.). Layouts in Flutter. https://docs.flutter.dev/ui/layout

Flutter. (s. f.). Layout widgets. https://docs.flutter.dev/ui/widgets/layout

Flutter. (s. f.). Understanding constraints. https://docs.flutter.dev/ui/layout/constraints

Flutter. (s. f.). Material components. https://docs.flutter.dev/ui/widgets/material

Flutter. (s. f.). CustomPaint class. https://api.flutter.dev/flutter/widgets/CustomPaint-class.html

Flutter. (s. f.). Hero animations. https://docs.flutter.dev/ui/animations/hero-animations

Flutter. (s. f.). Use a custom font. https://docs.flutter.dev/cookbook/design/fonts

Flutter. (s. f.). Build a form with validation. https://docs.flutter.dev/cookbook/forms/validation

Google. (s. f.). Material Design 3. https://m3.material.io

Google Fonts. (s. f.). Lilita One. https://fonts.google.com/specimen/Lilita+One

Google Fonts. (s. f.). Poppins. https://fonts.google.com/specimen/Poppins

Gordillo Mendoza, J. A., y Escobedo Ocaña, J. L. (s. f.). MenúRadar: aplicativo móvil universitario para la gestión en tiempo real de menús estudiantiles [Trabajo del curso, PE Lab 03, segunda unidad]. Universidad Nacional de San Agustín de Arequipa.